Adamson University College of Science
IT and IS Department

tcha

Course Code	ITCD 103	Section	29104
Course Details	Software Development for Enterprise Systems	Date	03/28/26
Names	Fernandez, Aaron Clifford Gardoce, Allan Mark Portes, John Dale Salazar, Kaede Sotelo, Andria Faye Vegerano, Miguel Joseph	Hands-On Laboratory	

SDLC Phase 2
Hands-on Laboratory Exercise 2

Objective: The objective of Phase 2 Design in the Software Development Life Cycle (SDLC) for the Hands-on Laboratory project is to do the following:

1 Create a comprehensive and detailed blueprint outlining the laboratory system's architecture, functionality, and technical specifications.
2 Translate the requirements gathered in Phase 1 into a tangible design plan that will guide the development team in implementing the laboratory solution efficiently and effectively.
3 Define the precise boundaries of your project, what it will deliver, and what it won't include.

1.1 Project Idea:
1.2 Tasks:
1.2.1 Refine Requirements:
Start by gathering and reviewing the requirements identified in Phase 1. This might include user stories and functional and non-functional requirements documents.

I. SMART Goals

Goal 1: System Development

Specific	Develop a fully functional Computer Repair Shop Management System using Salesforce that manages customer records, repair tickets, status tracking, and service history.
Measurable	System must successfully process at least 50 test repair tickets with 100% data accuracy and zero critical bugs.

Achievable	Utilize Salesforce Developer Edition (free) with team expertise in cloud computing and database relationships developed through coursework.
Relevant	Addresses the real business need of local repair shops currently using inefficient manual or spreadsheet-based systems.
Time-bound	Complete initial development and testing within 3–4 weeks.

Goal 2: User Adoption

Specific	Create an intuitive system interface that repair shop staff can use without extensive technical training.
Measurable	Achieve 80% user satisfaction rating and enable staff to complete common tasks (creating repair tickets, updating status, searching customer history) in under 2 minutes.
Achievable	Design interfaces based on standard Salesforce layouts familiar to users, supplemented with custom training documentation.
Relevant	System usability directly impacts adoption rates and determines whether shops will transition from current methods.
Time-bound	Conduct user testing and achieve satisfaction targets within the 3–4 week development timeline.

Goal 3: Data Security Compliance

Specific	Implement security measures that fully comply with the Data Privacy Act of 2012 and protect sensitive customer information.
Measurable	Pass security audit checklist with 100% compliance on encryption, access controls, and data handling procedures.
Achievable	Leverage Salesforce's built-in security features including user authentication, role-based permissions, and encrypted data storage.
Relevant	Protects customer privacy and shields repair shops from legal liability related to data breaches.
Time-bound	Security implementation and validation completed by Week 3 of the development timeline.

II. Functional Requirements

The system must fulfill the following functional capabilities:

Functional Requirement	Description
Customer Management	Create, update, and search customer records with complete service history.
Repair Ticket Management	Generate unique ticket IDs, record device information, and track repair statuses.
Status Tracking	Update repair status in real time (e.g., Received, In Progress, Awaiting Parts, Completed, Released) with automated email notifications sent to customers upon completion.

Service History	Maintain complete repair history accessible from customer profiles.
Reporting and Analytics	Display dashboard metrics and generate monthly performance reports.
User Authentication	Implement secure login with role-based permissions for Admin, Technician, and Reception roles.

III. Non-Functional Requirements

The system must meet the following performance and quality standards:

Quality Standard	Requirement
Performance	System response time under 3 seconds, supporting up to 5 concurrent users.
Reliability	99% uptime during business hours with automatic Salesforce cloud backup.
Security	Data encryption, password complexity requirements, and Data Privacy Act of 2012 compliance.
Usability	Intuitive interface requiring only 2-hour training for basic computer users.
Maintainability	Documented configuration with point-and-click customization capability.
Scalability	Support for multiple shop locations and growth to 10,000+ customer records.

IV. User Needs and Target Audience

The system targets four primary user groups:

- Shop Owners need visibility into operations, financial reports, and data-driven insights for business decisions. Current manual systems provide no analytics, making it difficult to track revenue sources or customer retention.
- Receptionists require quick customer lookup, simple ticket creation, and immediate status updates. Paper logs make searches time-consuming and prevent systematic customer notifications.
- Technicians need clear views of assigned tickets, documentation space for technical findings, and quick status updates without workflow interruption. Current systems lack standardized repair documentation formats.
- Customers need transparency regarding repair status, timely pickup notifications, and secure data handling. The frustration from lack of updates currently requires frequent phone calls to shops.

The target audience consists of small to medium-sized local computer repair shops (1–5 employees) currently using manual or spreadsheet-based systems, located in areas with stable internet connectivity, seeking to professionalize operations without high software costs.

V. User Stories

The following user stories represent specific scenarios from the perspective of end users:

ID	Role	User Story
US1	Receptionist	As a receptionist, I want to quickly search for a customer by phone number so that I can retrieve their information when they call about their repair status.
US2	Technician	As a technician, I want to update the repair status to "Awaiting Parts" and add a note explaining which parts are needed so that the customer can be informed of the delay.
US3	Shop Owner	As a shop owner, I want to view a dashboard showing how many repairs are in progress, completed this week, and total revenue so that I can monitor business performance.

US4	Customer	As a customer, I want to receive an email notification when my laptop repair is completed so that I know when to pick it up without calling the shop.
US5	Receptionist	As a receptionist, I want to create a new repair ticket by entering device information and issue description so that the repair can be tracked from start to finish.

Summary

This requirements definition establishes a clear foundation for the Computer Repair Shop Management System. The SMART goals provide measurable targets, functional and non-functional requirements specify system capabilities and quality standards, and user stories ensure the solution addresses real user needs. These requirements will guide subsequent development phases and serve as acceptance criteria for project completion.

1.2.2 System Architecture Design:

Sketch out the overall system architecture and represent different modules (e.g., user login, data storage, search function).

System Architecture Overview

The proposed Computer Repair Shop Management System follows a cloud-based, multi-tier architecture built on the Salesforce platform. The architecture is designed to ensure scalability, security, and high availability while minimizing infrastructure management.

The system is divided into three primary layers:

1. Presentation Layer (User Interface)

This layer handles all user interactions and is built using Lightning Web Components (LWC).

Components:
- Login Interface
- Dashboard (Owner View)
- Repair Ticket Forms (Receptionist)
- Technician Workspace
- Customer Record Pages

Responsibilities:
- Render dynamic UI
- Capture user input
- Communicate with backend via Apex controllers

2. Application Layer (Business Logic)

This is where the system actually becomes "smart."

Technologies:
- Apex (Backend Logic)
- Triggers
- SOQL (Database Queries)

Components:
- Customer Management Logic
- Repair Ticket Processing
- Status Automation (e.g., auto email when completed)
- Role-Based Access Control

Responsibilities:
- Enforce business rules
- Process transactions
- Validate inputs
- Automate workflows

Example Flow:
Receptionist creates ticket → Apex validates → Trigger assigns ID → Record saved
Technician updates status → Trigger fires → Email notification sent

3. Data Layer (Database)

Handled entirely by Salesforce's multi-tenant cloud database.

Core Entities:
- Customers
- Repair Tickets
- Users (Admin, Technician, Receptionist)
- Service History

Features:
- Automatic backup
- Data encryption
- Relationship mapping (Lookup / Master-Detail)

The system leverages Salesforce's built-in relational database structure, eliminating the need for manual database configuration while ensuring ACID compliance.

4. Integration Layer (APIs & External Services)

Technologies:
- Salesforce REST API
- Email-to-Case

Responsibilities:

- Enable external communication
- Automate email notifications
- Allow future integrations (mobile apps, analytics tools)

Data Flow

Example Data Flow:

1. User logs in → Authentication handled by Salesforce
2. Receptionist creates repair ticket → LWC → Apex Controller
3. Apex validates + processes → SOQL inserts data
4. Trigger executes → Generates Ticket ID + updates status
5. If status = "Completed" → Email-to-Case sends notification
6. Data reflected in dashboard in real-time

Architecture Style Justification

The system uses a **Platform-as-a-Service (PaaS) architecture**, which eliminates the need for:

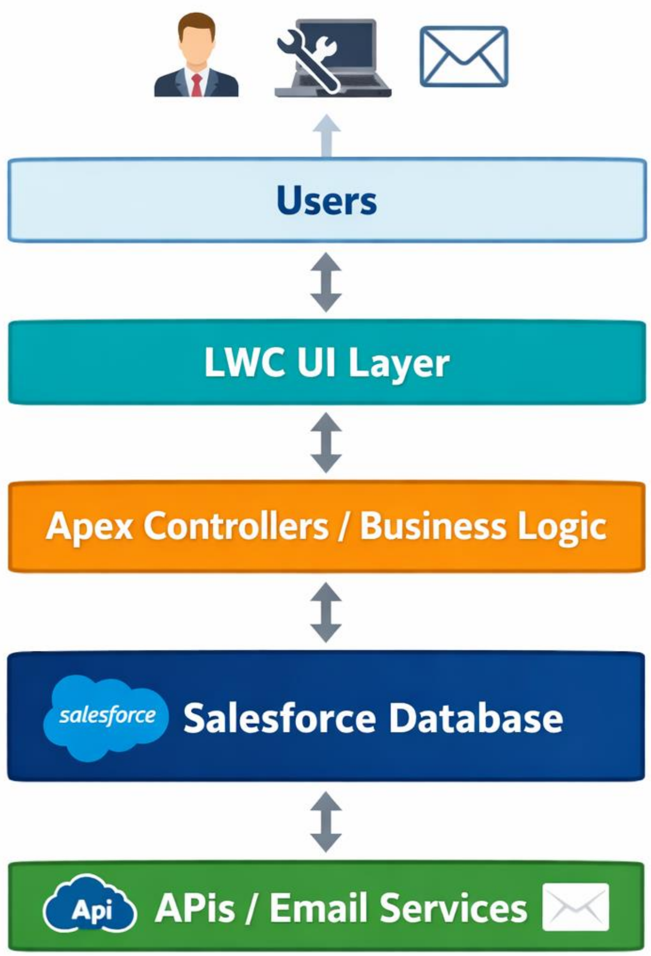
- Server management
- Manual database setup
- Backend infrastructure

Why this is the best choice:

- Faster development (matches capstone constraint)
- Built-in security (Data Privacy Act compliance)
- High reliability (Salesforce SLA)
- Scalable without redesign

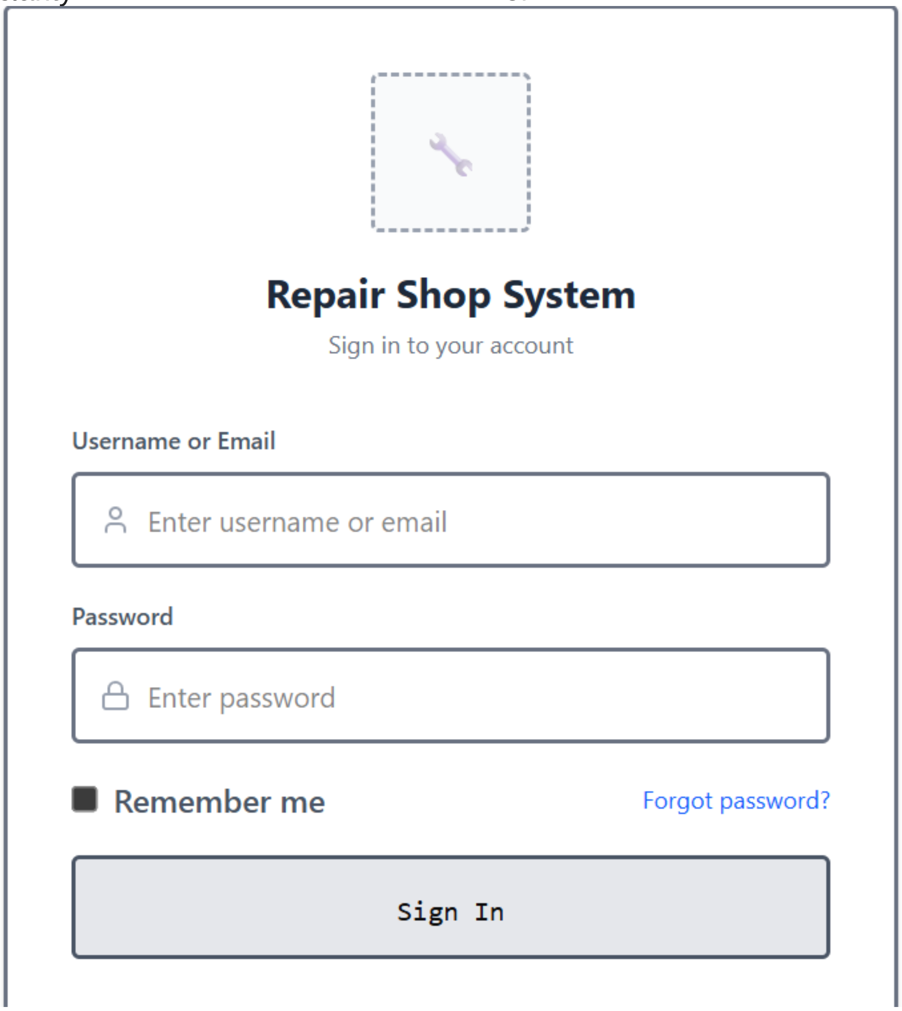
Adamson University College of Science
IT and IS Department

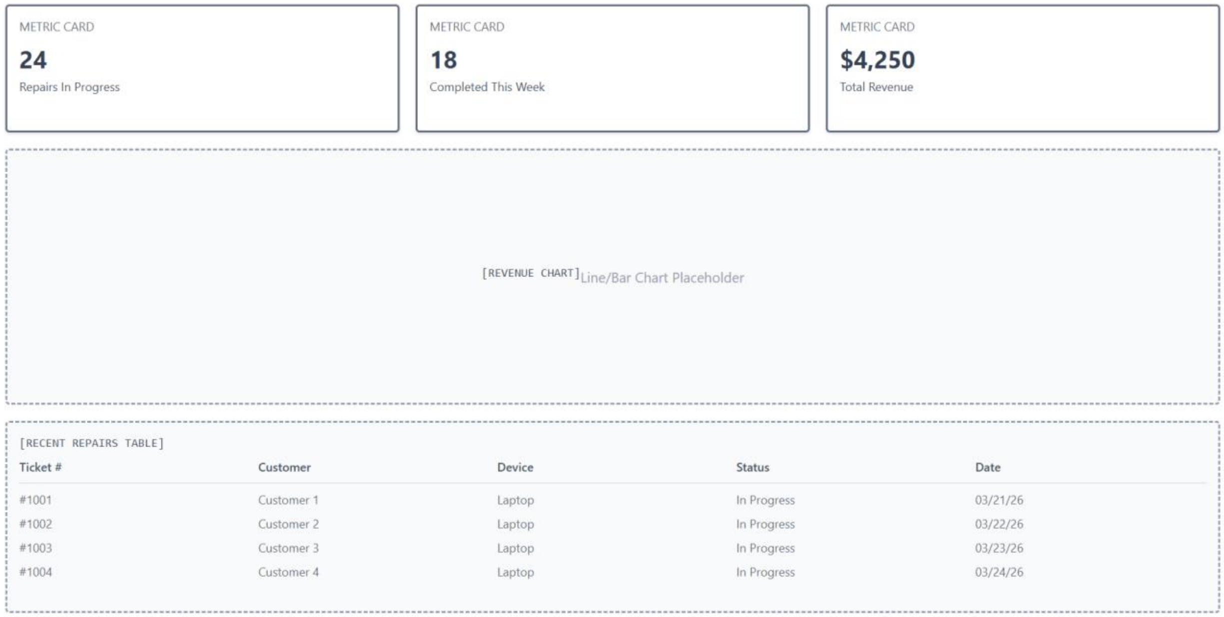
System Architecture Diagram

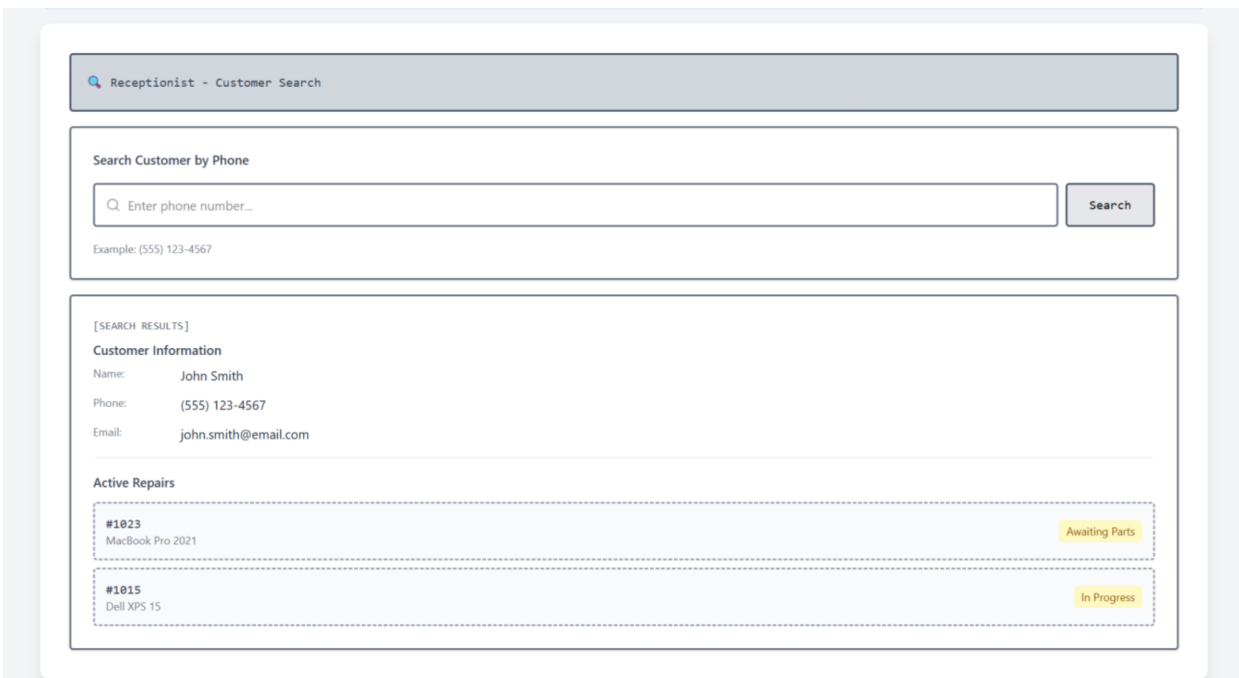


1.2.3 User Interface (UI) Design:

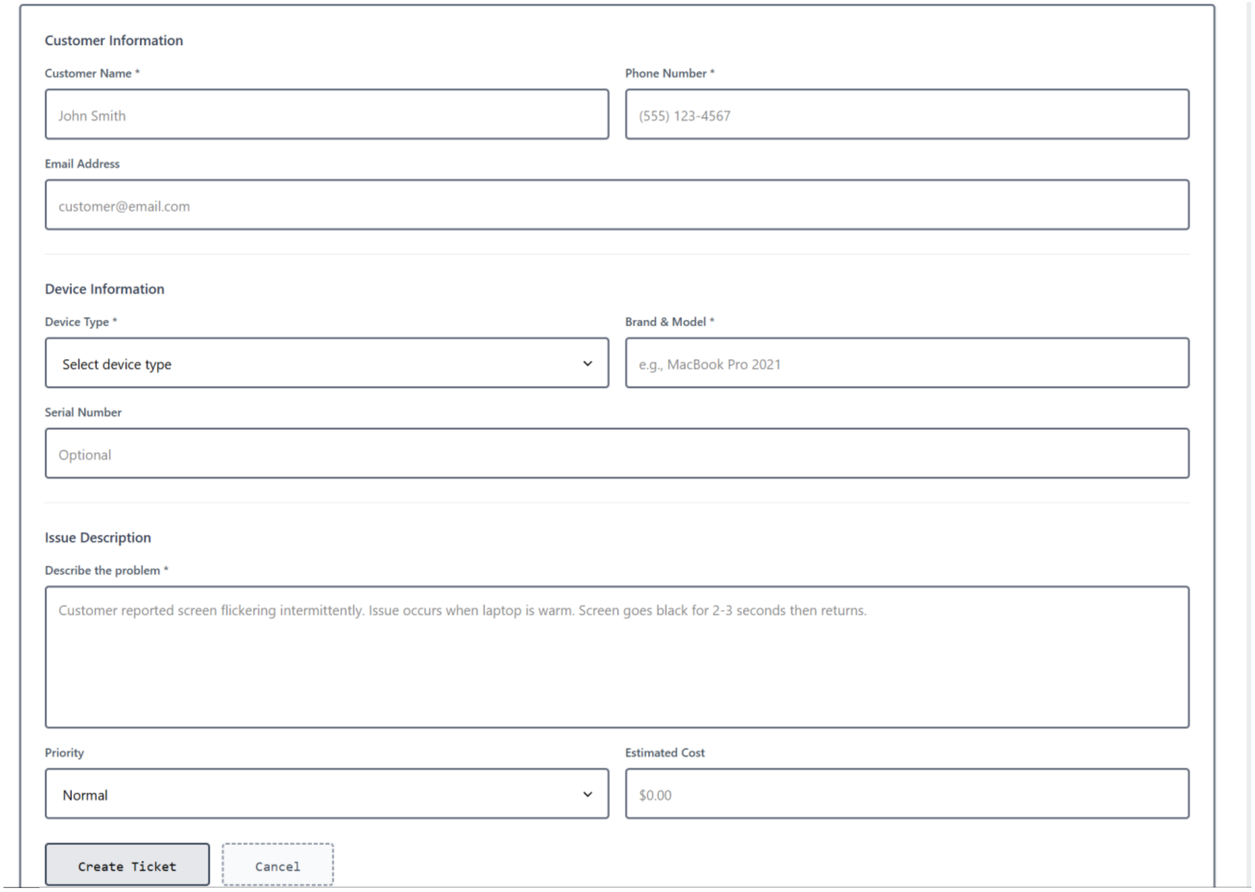
Sketch wireframes for critical screens of your project (e.g., login screen, main dashboard). Consider user experience (UX) principles like ease of navigation and clarity of information.




METRIC CARD
24
Repairs In Progress
METRIC CARD
18
Completed This Week
METRIC CARD
$4,250
Total Revenue
[REVENUE CHART]Line/Bar Chart Placeholder
[RECENT REPAIRS TABLE]
Ticket # | Customer | Device | Status | Date
#1001 | Customer 1 | Laptop | In Progress | 03/21/26
#1002 | Customer 2 | Laptop | In Progress | 03/22/26
#1003 | Customer 3 | Laptop | In Progress | 03/23/26
#1004 | Customer 4 | Laptop | In Progress | 03/24/26


Receptionist - Customer Search
Search Customer by Phone
Enter phone number...
Search
Example: (555) 123-4567
[SEARCH RESULTS]
Customer Information
Name: John Smith
Phone: (555) 123-4567
Email: john.smith@email.com
Active Repairs
#1023
MacBook Pro 2021
Awaiting Parts
#1015
Dell XPS 15
In Progress

Adamson University College of Science
IT and IS Department



Customer Information

Customer Name *

John Smith

Phone Number *

(555) 123-4567

Email Address

customer@email.com

Device Information

Device Type *

Select device type

Brand & Model *

e.g., MacBook Pro 2021

Serial Number

Optional

Issue Description

Describe the problem *

Customer reported screen flickering intermittently. Issue occurs when laptop is warm. Screen goes black for 2-3 seconds then returns.

Priority

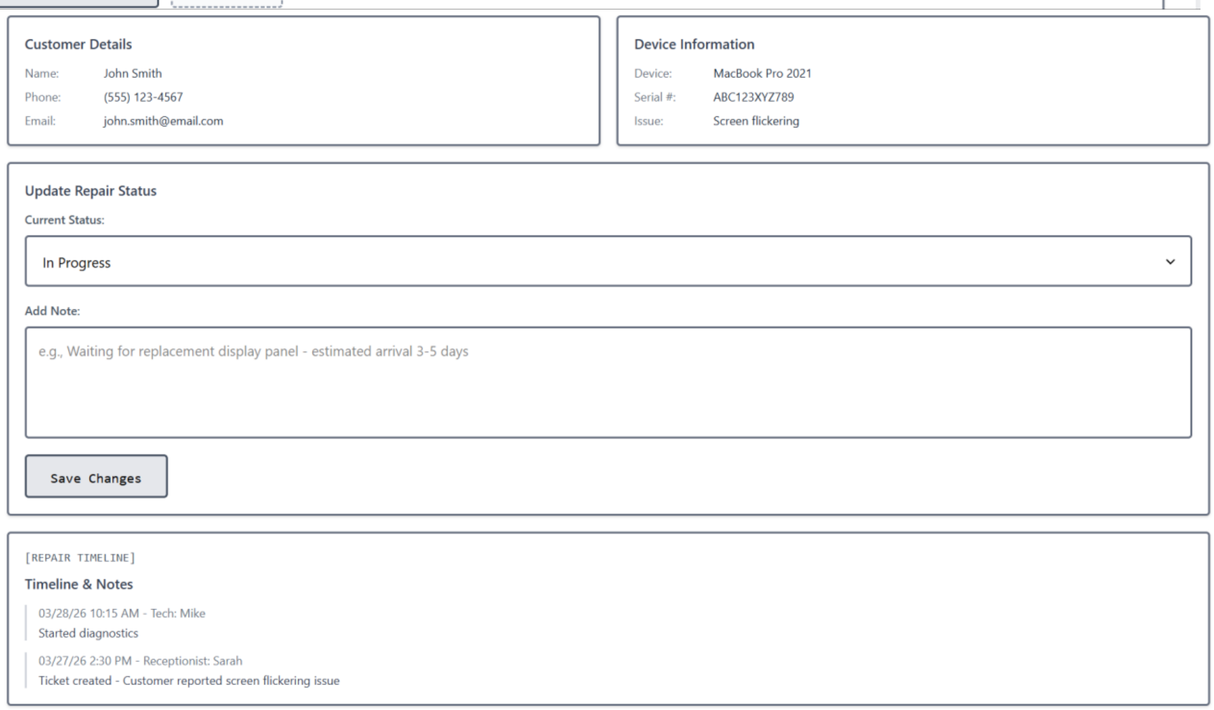
Normal

Estimated Cost

$0.00

Create Ticket

Cancel



Customer Details

Name: John Smith

Phone: (555) 123-4567

Email: john.smith@email.com

Device Information

Device: MacBook Pro 2021

Serial #: ABC123XYZ789

Issue: Screen flickering

Update Repair Status

Current Status:

In Progress

Add Note:

e.g., Waiting for replacement display panel - estimated arrival 3-5 days

Save Changes

[REPAIR TIMELINE]

Timeline & Notes

03/28/26 10:15 AM - Tech: Mike

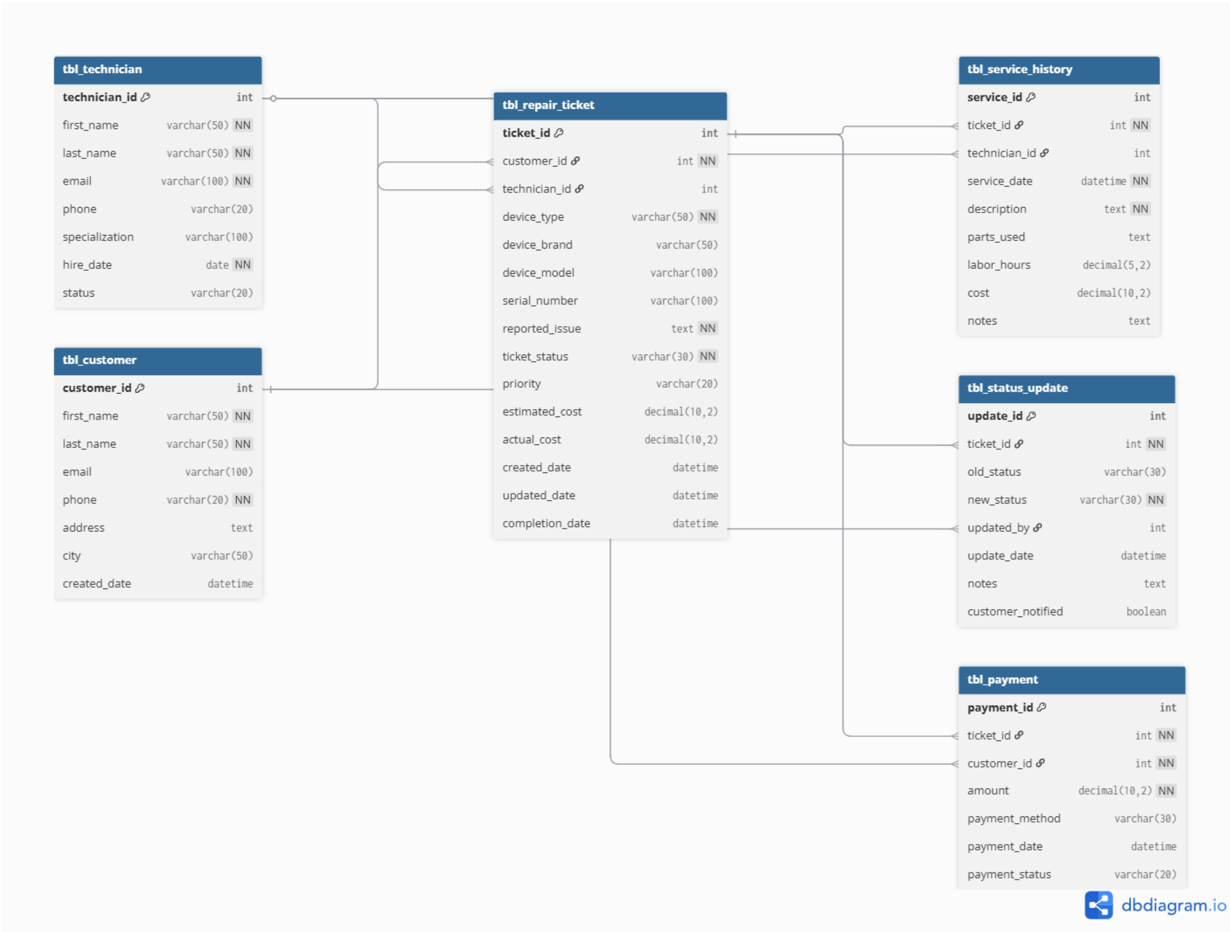
Started diagnostics

03/27/26 2:30 PM - Receptionist: Sarah

Ticket created - Customer reported screen flickering issue

1.2.4 Database Design:

Create an Entity-Relationship Diagram (ERD) to represent the data entities in your system and their relationships.



1.2.5 Detailed Design:

Choosing programming languages, frameworks, and APIs (Application Programming Interfaces) you'll use.

Components	Description	Advantages	Design Rationale
Programming Language: Apex	An object oriented 4th generation language designed for executing flow and transaction control statements on the salesforce platform.	This language enables the automation of the repair lifecycle through custom	The team selected Apex to satisfy the Principles of OOP, as its Java-like syntax allows for the

		triggers, ensuring 100% data integrity for every customer record.	application of encapsulation and inheritance in the backend logic.
Framework: Lightning Web Components (LWC)	Lightning Web Components (LWC) is a modern UI framework built on standard web stack components including HTML, CSS, and modern JavaScript.	The use of LWC provides an intuitive, responsive interface that maintains page load speeds under three seconds for shop technicians.	This framework was chosen to fulfill the application of web development technologies by using the industry standard scripting languages.
API's: Salesforce REST API & Email-to-Case	The system utilizes the Salesforce REST API for web service integration and SOQL for direct database manipulation	The combination facilitates automated email-to-case intake and real-time status updates, allowing the system to scale efficiently as the business grows.	These tools were integrated to meet the exploration of Database Management Systems, demonstrating the team's ability to design and query relational databases in a real-world scenario.

1.2.6 Design Review:

Give the architecture details, UI mockups, data flow, and technical specifications.

The design review summarizes and evaluates the overall system design to ensure that all components work together effectively and meet the project requirements.

Based on Refine Requirements, the system is designed to support key features such as customer management, repair ticket tracking, status updates, and reporting. The SMART goals and user stories guided the design to ensure the system is functional, user-friendly, and secure. System Architecture Design, the system follows a modular structure using the Salesforce platform. It includes components such as user authentication, customer management, repair ticket processing, and reporting modules. These components interact with the database and APIs to ensure smooth data flow and system performance.

In terms of User Interface (UI) Design, the system provides simple and intuitive screens such as login pages, dashboards, and forms for ticket creation. The UI is designed to be easy to navigate, allowing users like receptionists and technicians to complete tasks quickly and efficiently. For Database Design, the Entity-Relationship Diagram (ERD) defines key entities such as Customers, Repair Tickets, Users, and Service History. These entities are connected to ensure organized data storage, easy retrieval, and data consistency across the system. Lastly the Detailed Design specifies the technologies used, including Apex for backend logic, lLightning Web Components (LWC) for the frontend, and Salesforce REST APIs for integration. These technologies ensure system reliability, scalability, and security.

Overall, the design review confirms that the system architecture, interface, data structure, and technical components are well-aligned with the project goals and are ready for implementation.

1.3 Deliverables:

- System Architecture Diagram
- UI Wireframes
- Data Model
- Technical Design Specifications
- Software Design Document

Conclusion:

In conclusion, we successfully completed the Design Phase of the Software Development Life Cycle (SDLC) by transforming the gathered requirements into a clear and structured system design. Through refining the requirements, we were able to better understand the needs of the users and define the system's goals and functionalities. We have created the system architecture, UI design, database, and technical specifications, which serves as a strong blueprint for the development process. Each component was carefully planned to ensure the system is efficient, user-friendly, secure, and scalable.

Overall, this laboratory activity helped us improve our skills in planning, analyzing, and organizing system components, which are essential in developing real-world software solutions.